Numerical Methods for Differential Equations in Model Rocket Simulation

Boula Etans
Department of Mathematics, Riverside City College

Faculty Mentor: Dr. Mary Margarita Legner
Professor of Mathematics, Riverside City College

Presented at the 26th Annual Building Bridges Research Conference,
Honors Transfer Council of California, March 28, 2026

Presented at the 18th Annual RCCD Student Research Conference,
Riverside Community College District, November 21, 2025

Code and data available at `https://github.com/boulaetans/numericalrocketry`

Table of Contents

Abstract

Since the dawn of the space age, numerical methods have played a defining role in rocket development by enabling engineers to model trajectories long before physical testing. Because many governing equations in flight dynamics lack closed-form solutions (Karbon, 1998), numerical methods are the practical way to analyze and predict behavior. This project applies those techniques to small-scale rocket flight, extending a previously verified 1D ascent simulator into a full 3D Python-based flight simulation with explicit geometry, event-aware dynamics, and atmospheric modeling.

The rocket's geometry is defined component-wise (nose, body tubes, transitions, and fins) with full parametric dimensions, from which reference and wetted areas are computed. Atmospheric conditions follow the International Standard Atmosphere (ISA) temperature and pressure model, enabling Mach and Reynolds number corrections, with angle of attack computed against a wind model, validated at zero wind to match the actual launch conditions. Aerodynamic forces follow a Barrowman-derived approach for normal force and center of pressure, including nose pressure drag, fin and body contributions, base drag, and transonic corrections consistent with model-rocket practice (Niskanen, 2013). Three-dimensional attitude is represented using quaternions, which rotate aerodynamic and thrust vectors between the body and inertial frames, and translational and rotational dynamics are integrated using a fixed-stage fourth-order Runge-Kutta (RK4) scheme combined with adaptive step-size control that shrinks the timestep around ignition, burnout, and other rapidly changing force events and relaxes it during smoother coasting. This scheme was chosen over the Euler and classical predictor-corrector methods used earlier for its better handling of discontinuities and rapidly changing forces during ascent (Derrick and Grossman, 1997; Bryan, 2025).

The prior 1D model reached apogee within 2.00% of OpenRocket's prediction, with its three integrators (Euler, Adams-Bashforth-Moulton, and RK4) agreeing with each other to within 0.1%; this motivated its use as a baseline benchmark as the simulator expanded into 3D flight, and it has since been retired now that the 3D model is independently validated. The completed 3D model was validated against OpenRocket's own simulation of the identical rocket, motor, and launch conditions, and against flight-computer data recorded from an actual launch of the physical rocket. Apogee altitude, apogee time, and peak velocity agreed with OpenRocket to within 0.01%, roughly a 200-fold reduction in error relative to the 1D baseline. Comparison against the real flight showed a larger gap, with recorded apogee about 12.42% higher than either simulation. Because the two independently built simulators agreed with each other too closely for a shared modeling error to explain a gap that size, this was traced to the real flight's own motor and sensor variance: the flight computer's barometric altimeter had no accelerometer, and its raw log required correcting a ground-elevation error, removing duplicate samples, and re-deriving velocity and acceleration from pressure data with a Kalman filter before a fair comparison was possible.

Beyond its computational contribution, this project shows how undergraduate students can apply numerical methods for differential equations to build and validate a complete engineering simulation against a reference simulator and a real physical flight, turning abstract mathematics into a practical, tested tool and a foundation for future research in rocket flight dynamics. Numerical modeling is especially valuable in undergraduate engineering, giving students an accessible way to explore real-world systems through mathematics and computation (Niskanen, 2013).

Keywords: numerical methods, ordinary differential equations, Runge-Kutta method, rocket flight simulation, model validation, Kalman filter

1. Introduction and Background

This project began as an honors contract requirement for MAT-2 (Differential Equations) at Riverside City College. Many governing equations in flight dynamics, including the coupled translational and rotational equations of motion for a rigid body under aerodynamic, thrust, and gravitational loading, lack closed-form analytical solutions. Numerical methods exist precisely to analyze and predict the behavior of systems like this, and this project applies those techniques to small-scale, amateur model rocket flight.

The project has two phases. The first phase, preserved in the codebase's `legacy/` directory, is a 1D ascent-only numerical-methods comparison: three classical integrators (Euler, second-order Adams-Bashforth-Moulton, and fourth-order Runge-Kutta) applied to a simplified single-axis model of the same real rocket and motor. This phase served its purpose as an initial numerical-methods exercise and as a baseline benchmark, but it has no attitude dynamics, no exact Barrowman center-of-pressure treatment, and no recovery phase.

The second and primary phase extends that foundation into a full 3D, six-degree-of-freedom (6DOF) Python flight simulator, independently reimplementing the rocket-flight physics used by the open-source simulator OpenRocket by studying its published methodology and validating results directly against it. Unlike the 1D model, the 3D simulator represents the rocket's real geometry component-by-component, computes an explicit stability margin, models off-axis and rotational motion, and simulates the full flight envelope from ignition through parachute recovery and touchdown. This report documents the physical and numerical methods behind that simulator in full mathematical detail, presenting the exact governing equations used at every stage of the pipeline, and presents its validation against both OpenRocket's reference simulation and a real logged flight of the physical rocket.

The simulator is not a general-purpose tool: its geometry, mass table, and motor are hardcoded to one specific real rocket, nicknamed "Green Eggs," flying a real Estes C11-5 motor, launched and logged as part of this project. This deliberate specialization, one rocket, one motor, one launch, is what makes an exhaustive, line-item validation against two independent references possible: every numerical result in Section 5 traces back to a specific, fixed physical configuration rather than a swept design space.

1.1 Why a Full Rewrite Was Necessary

The 1D model integrates a single scalar equation of motion,

$$m(t)\,\dot{v} = T(t) - \tfrac{1}{2}\rho(h)\,C_D(v,h)\,S_{\text{ref}}\,v\,|v| - m(t)\,g, \tag{1}$$

where v is vertical velocity, h altitude, $m(t)$ the instantaneous (propellant-depleting) mass, $T(t)$ thrust from the motor's measured curve, and C_D a zero-lift drag coefficient. Equation 1 has no angle of attack, no restoring moment, and no stability margin, so it has no way to represent the rocket tipping, weathercocking into wind, or oscillating in pitch: it assumes the rocket flies perfectly vertically at zero angle of attack for its entire flight. It also has no parachute or descent model, so it can only be validated against the ascent phase of a flight. Reproducing OpenRocket's own behavior, which explicitly tracks orientation, angular velocity, and center-of-pressure migration as propellant burns, required promoting the state vector from a single scalar (h, v) pair to a full 13-dimensional rigid-body state (3 position, 3 velocity, 4 quaternion components, 3 angular velocity components) evolved under Newton-Euler dynamics, described in Section 3.

2. Vehicle Configuration

All geometry, mass, and motor values below are taken directly from the rocket's OpenRocket design file (`green_eggs_openrocket_design.ork`), the same design file used to generate the OpenRocket reference simulation this project validates against, not estimated or approximated.

Table 1: *Green Eggs Airframe Geometry*

Property	Value
Overall Length	571.5 mm
Body Diameter	45.62 mm
Nose Cone Type	Elliptical, 136.5 mm long
Fin Count	3
Fin Planform	Root chord 71.12 mm, tip chord 24.13 mm, span 68.58 mm, sweep 76.2 mm
Fin Thickness	2.46 mm, rounded cross-section
Launch Rail Length	0.9144 m (3 ft), 0° rail angle
Dry Mass (No Motor)	130.7 g
Liftoff Mass (With Motor)	166 g
Motor	Estes C11-5 (24 mm, 70 mm, 5 s ejection delay)
Recovery	18 in (0.4572 m) parachute, $C_d = 0.8$, deployed on motor ejection charge
Launch Site	532 m MSL, 33.98° N latitude

2.1 Component-Wise Mass Table

Every dry component (nose cone, body tubes, tube coupler, avionics bay, fin set, centering rings, parachute, shock cord, launch lug, and others) is modeled individually with its own mass, axial position, and, where relevant, wall thickness, rather than as a single lumped mass. Table 2 reproduces the complete component list used by the simulator's mass model, with axial positions measured from the nose tip. These values were independently re-derived from the design file's raw XML offset formulas and confirmed to match to the last digit.

Table 2: *Dry Component Mass Table (from* `rocket_config.py`*)*

Component	Mass (g)	x_{start} (mm)	Length (mm)	Radius (mm)
Nose Cone (shell)	21.83	0.0	136.5	22.81
Body Tube (forward)	11.34	136.5	127.0	22.81
Tube Coupler	17.01	238.1	63.5	22.81
Apogee egg protector	1.00	170.0	60.0	20.32
EasyMini 2 (flight computer)	6.50	181.0	38.0	10.00
Battery	9.00	181.6	36.9	13.25
Body Tube (bulkhead segment)	0.91	263.5	3.2	22.81
Body Tube (aft)	28.92	266.7	304.8	22.81
Freeform Fin Set (3 fins)	12.47	471.2	100.3	22.81
Inner Tube (motor mount)	1.59	501.7	69.9	12.40
Engine hook	1.13	509.3	72.4	1.52
Centering Ring (motor mount)	0.75	501.7	6.4	12.04
Inner Tube (aft)	0.23	535.9	12.7	13.14
Qualman Baffle	1.00	406.4	63.5	21.59
Parachute (canopy plus lines)	7.47	309.9	38.1	22.23
Shock Cord	7.94	353.1	25.0	21.59
Centering Ring (x2)	0.74 each	517.1 / 562.9	1.0	22.23
Launch Lug	0.17	368.3	50.8	2.88
Motor dry mass (case)	23.30	(motor-mount-referenced, see §2.2)		12.00

Summed, these dry components total 130.7 g, matching the reference simulation's own recorded dry mass. Adding the C11's rated propellant mass (12.0 g) gives the 166 g liftoff mass used to initialize every simulation run in this project.

2.2 Motor Mounting Offset

The motor mount tube does not center the motor: the design file specifies a 6.35 mm overhang, meaning the motor sits 6.35 mm past the aft end of its mount tube. If x_{mount} is the mount tube's own start position and L_{mount}, L_{motor} are the mount and motor lengths, the motor's leading-edge position is

$$x_{\text{motor}} = x_{\text{mount}} + (L_{\text{mount}} - L_{\text{motor}} + \delta_{\text{overhang}}), \tag{2}$$

with $\delta_{\text{overhang}} = 6.35$ mm, and the motor's own center of gravity (a RASP-standard motor is always assumed uniform, so its CG sits at its geometric midpoint) is $x_{\text{motor}} + \frac{1}{2}L_{\text{motor}}$. This single offset shifts the loaded rocket's CG by several millimeters relative to a naive centered-motor assumption and is applied consistently to both the dry motor casing mass and the depleting propellant mass during flight.

2.3 Shape-Integrated Nose Cone and Fin Mass Models

The nose cone and fin set are both structurally non-solid but are not simple hollow cylinders, so they use dedicated shape-integrated mass models rather than the solid or hollow-cylinder approximation used for the tube-shaped components. Getting these two models right was necessary to bring the simulator's predicted center of gravity within 0.5% of OpenRocket's own, since every other validation metric in this report depends on that mass model being correct.

Nose cone shell centroid

The nose cone is treated as a hollow shell of constant wall thickness t_w following the outer profile

$r(x)$ (Section 3.3 gives the exact profile equations by shape). The nose is discretized into $N = 128$ axial frustum slices of width $\Delta x = L_{\text{nose}}/N$. For each slice with outer radii r_{1o}, r_{2o} at its two faces, the local shell thickness projected along the slope is

$$h_{\text{shell}} = t_w \cdot \frac{\sqrt{(r_{2o} - r_{1o})^2 + \Delta x^2}}{\Delta x}, \tag{3}$$

giving inner radii $r_{1i} = \max(r_{1o} - h_{\text{shell}}, 0)$, $r_{2i} = \max(r_{2o} - h_{\text{shell}}, 0)$. Each slice's shell volume is the difference of two frustum volumes ($V_{\text{frustum}} = \frac{\pi}{3}\Delta x(r_1^2 + r_1 r_2 + r_2^2)$) and its own centroid is the standard conical frustum centroid formula

$$\bar{x}_{\text{frustum}} = \frac{\Delta x\,(r_1^2 + 2r_1 r_2 + 3r_2^2)}{4\,(r_1^2 + r_1 r_2 + r_2^2)}. \tag{4}$$

Summing the volume-weighted centroids of all 128 shell slices, then adding the rear shoulder (a short constant-thickness ring, plus an end cap if capped) with its own volume and centroid, gives the shell's true mass-weighted centroid. Because more shell material naturally sits toward the wider (aft) end of a tapered profile, this centroid is not at $0.5\,L_{\text{nose}}$ for any non-cylindrical shape, the exact opposite of the simplifying assumption a lumped-mass model would make.

Fin set centroid

Each fin's mass-CG is a mass-weighted blend of three physically distinct pieces:

- **Bulk planform:** the visible fin polygon's true area centroid, computed exactly via the shoelace formula

$$\bar{x}_{\text{bulk}} = \frac{1}{6A}\sum_i (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i), \qquad A = \tfrac{1}{2}\left|\sum_i (x_i y_{i+1} - x_{i+1} y_i)\right|, \tag{5}$$

  extruded by the fin thickness and a cross-section relative-volume factor (0.99 for Green Eggs' rounded cross-section).

- **Through-the-wall tab:** a rectangular tab passing through the airframe wall, with no relative-volume factor applied since it is not an exposed aerodynamic surface.

- **Root fillet:** the glue fillet running the fin's full swept axial extent, in its own (generally different-density) fillet material, with cross-sectional area found from the fillet radius r_f and body radius r_b via

$$A_{\text{fillet}} = 2\left[r_f^2 \tan(\theta_o) - \tfrac{1}{2}r_f^2\theta_o - \tfrac{1}{2}r_b^2\theta_i\right], \quad \theta_i = \arcsin\frac{r_f}{r_f + r_b}, \quad \theta_o = \arccos\frac{r_f}{r_f + r_b}. \tag{6}$$

The final per-fin centroid is $\bar{x}_{\text{fin}} = (m_{\text{bulk}}\bar{x}_{\text{bulk}} + m_{\text{tab}}\bar{x}_{\text{tab}} + m_{\text{fillet}}\bar{x}_{\text{fillet}})/(m_{\text{bulk}} + m_{\text{tab}} + m_{\text{fillet}})$. A design-file mass override on the fin set (Green Eggs' file documents one, labeled "paint and glue fillet allowance") replaces only the total reported weight of the assembly, never this shape-integrated position; the override and the position calculation are independent of each other.

2.4 Inertia Tensor

For a rigid body with x as the roll (symmetry) axis, each component contributes its own inertia about its own centroid plus a parallel-axis correction for its offset $dx = x_{\text{cg,i}} - x_{\text{cg,total}}$ from the

vehicle's total CG:

$$I_{xx} = \sum_i I_{xx,i}^{\text{own}}, \tag{7}$$

$$I_{yy} = I_{zz} = \sum_i \left(I_{\text{transverse},i}^{\text{own}} + m_i \, dx_i^2 \right). \tag{8}$$

Every component is modeled as either a solid cylinder or a hollow tube depending on its declared wall thickness. For a solid cylinder of mass m, radius r, length L:

$$I_{xx}^{\text{own}} = \tfrac{1}{2} m r^2, \qquad I_{\text{transverse}}^{\text{own}} = \tfrac{1}{12} m \left(3r^2 + L^2 \right). \tag{9}$$

For a hollow tube of outer radius r_o and inner radius $r_i = r_o - t_w$:

$$I_{xx}^{\text{own}} = \tfrac{1}{2} m \left(r_o^2 + r_i^2 \right), \qquad I_{\text{transverse}}^{\text{own}} = \tfrac{1}{12} m \left[3 \left(r_o^2 + r_i^2 \right) + L^2 \right]. \tag{10}$$

Treating a genuinely tube-shaped part (body tubes, couplers, inner tubes, launch lug) as a solid cylinder instead of a hollow one would underestimate its own roll inertia by up to roughly a factor of two, since real tube mass sits at the rim rather than spread through the full cross-section. This is why `ComponentMass` carries an explicit `wall_thickness_m` field rather than assuming one shape for every component. Because the motor burns propellant over the course of the flight, this entire mass, CG, and inertia calculation is re-run at every RK4 sub-stage from the current instantaneous propellant mass, not computed once at liftoff; both the CG position feeding the static-margin calculation and the inertia tensor feeding the rotational equation of motion (Section 3.6) shift continuously as the motor burns.

3. Numerical Methods and Physics Model

The simulator advances a 13-component rigid-body state vector

$$\mathbf{y} = (\mathbf{p}_{\text{world}},\ \mathbf{v}_{\text{world}},\ \mathbf{q}_{\text{body}\to\text{world}},\ \boldsymbol{\omega}_{\text{world}},\ m_{\text{prop}}) \in \mathbb{R}^{3+3+4+3+1} \tag{11}$$

through time using a fixed-stage RK4 integrator with adaptive step-size control. This section documents, in order, the atmosphere model, the gravity model, the aerodynamic (Barrowman) model, the drag model, the rigid-body dynamics and quaternion attitude representation, and the time-integration scheme itself, matching the module structure of the `numericalrocketry.physics` package.

3.1 Atmosphere Model

Temperature and pressure

Atmospheric temperature follows a three-layer piecewise-linear International Standard Atmosphere (ISA) model as a function of altitude above mean sea level h:

$$T(h) = \begin{cases} T_0 + L_{\text{trop}}\, h, & 0 \le h \le 11{,}000 \text{ m} \\ 216.65 \text{ K}, & 11{,}000 < h \le 20{,}000 \text{ m} \\ 216.65 + L_{\text{strat}}\,(h - 20{,}000), & 20{,}000 < h \le 32{,}000 \text{ m} \\ 228.65 \text{ K}, & h > 32{,}000 \text{ m} \end{cases} \tag{12}$$

with sea-level reference temperature $T_0 = 288.15$ K, tropospheric lapse rate $L_{\text{trop}} = -0.0065$ K/m, and lower-stratosphere lapse rate $L_{\text{strat}} = +0.0010$ K/m. Pressure is integrated layer by layer from the hydrostatic barometric relation, using the power-law solution in each lapse-rate layer and the exponential solution in the isothermal tropopause layer:

$$P(h) = P_0 \left(\frac{T(h)}{T_0}\right)^{-g_0/(R\,L_{\text{trop}})}, \qquad 0 \le h \le 11{,}000 \text{ m}, \tag{13}$$

$$P(h) = P_{11}\, \exp\!\left(-\frac{g_0\,(h - 11{,}000)}{R \cdot 216.65}\right), \qquad 11{,}000 < h \le 20{,}000 \text{ m}, \tag{14}$$

$$P(h) = P_{20} \left(\frac{T(h)}{216.65}\right)^{-g_0/(R\,L_{\text{strat}})}, \qquad 20{,}000 < h \le 32{,}000 \text{ m}, \tag{15}$$

where $P_0 = 101{,}325$ Pa, $R = 287.053$ J/(kg K) is the specific gas constant for dry air, $g_0 = 9.80665$ m/s^2, and P_{11}, P_{20} are the pressures carried forward from the previous layer's boundary. This project's flight profile never exceeds about 116 m altitude, so only the lowest (tropospheric) branch is ever actually exercised in practice, but the full multi-layer model is implemented and validated so that the atmosphere function is correct at any altitude, matching the reference simulator's own general-purpose implementation.

Density, viscosity, and speed of sound

Air density follows directly from the ideal gas law,

$$\rho(h) = \frac{P(h)}{R\,T(h)}, \tag{16}$$

and dynamic viscosity follows Sutherland's law,

$$\mu(h) = \mu_0 \left(\frac{T(h)}{T_0}\right)^{3/2} \frac{T_0 + C_{\text{Suth}}}{T(h) + C_{\text{Suth}}}, \tag{17}$$

with $\mu_0 = 1.81 \times 10^{-5}$ Pa s and Sutherland's constant $C_{\text{Suth}} = 110.4$ K. Local speed of sound follows from the ISA temperature via the ideal-gas relation $a(h) = \sqrt{\gamma R T(h)}$, with $\gamma = 1.4$ for air, giving the Mach number directly as

$$M = \frac{|v_{\text{rel}}|}{a(h)} \tag{18}$$

for airspeed v_{rel} relative to the local wind. The Reynolds number used throughout the drag model is

$$\text{Re} = \frac{\rho(h)\,|v_{\text{rel}}|\,L_{\text{ref}}}{\mu(h)}, \tag{19}$$

using the rocket's overall length as the reference length L_{ref}, floored at $\text{Re} = 1$ to avoid a division-by-zero pathology at rest.

3.2 Gravity Model

The reference simulation this project targets defaults to the WGS84 ellipsoidal gravity model rather than a constant 9.80665 m/s^2: gravity varies about $\pm 0.5\%$ with latitude and falls off slightly with altitude, a real, if small, contributor to any timing or apogee gap against the reference flight. Latitude-dependent surface gravity follows the international gravity formula,

$$g_0(\phi) = 9.7803267714 \cdot \frac{1 + 0.00193185138639\,\sin^2\phi}{\sqrt{1 - 0.00669437999013\,\sin^2\phi}}, \tag{20}$$

where ϕ is geodetic latitude (33.98° N for the Green Eggs launch site, rounded from the design file's saved launch coordinates to about 1 km precision, which does not affect this formula's output at its native $\sim 0.01°$ sensitivity). This is then corrected for altitude above sea level using an inverse-square falloff with a spherical-Earth approximation,

$$g(h, \phi) = g_0(\phi) \left(\frac{R_\oplus}{R_\oplus + h}\right)^2, \qquad R_\oplus = 6{,}371{,}000 \text{ m}, \tag{21}$$

and this $g(h, \phi)$, not a constant, is what appears in the translational equation of motion (Section 3.6), evaluated fresh at the rocket's current mean-sea-level altitude (launch-pad AGL position plus the 532 m site elevation) at every RK4 sub-stage.

3.3 Aerodynamic Model: Barrowman Normal Force and Center of Pressure

Aerodynamic normal force and center of pressure follow a Barrowman-derived approach (Barrowman, 1967), computed separately for the nose cone and the fin set, combined with a nonlinear body-lift extension and a Mach-dependent body-fin interference correction, then summed.

Nose cone profile and pressure center

Slender-body theory gives a normal-force-coefficient slope of exactly $C_{N_\alpha,\text{nose}} = 2$ per radian for any axisymmetric nose shape, independent of profile. This is an exact result of Barrowman's linearized theory, not an approximation, and is why the same constant appears regardless of whether the nose is conical, ogival, or elliptical. What does depend on shape is where that normal force acts. Table 3

gives the nose center-of-pressure location x_{cp} (measured from the tip) used for each nose type the simulator supports; Green Eggs uses the elliptical case.

Table 3: *Nose Cone Center-of-Pressure Location by Shape*

Nose shape	x_{cp} (from tip, length L)
Conical	$\frac{2}{3}L$
Elliptical (quarter-ellipse profile)	$\frac{1}{3}L$
Parabolic (full parabola)	$\frac{1}{2}L$
Ogive / von Kármán (Haack, LD parameter 0)	$L - V_{\text{full}}/A_{\text{ref}}$ (exact volumetric CP)

The ogive/von Kármán case uses the exact volumetric-centroid identity $x_{\text{cp}} = L - V_{\text{full}}/A_{\text{ref}}$, where V_{full} is the nose's full solid-of-revolution volume, found not from a closed-form formula but by direct numerical quadrature of the profile. The nose is sliced into 128 frustums of width Δx, and

$$V_{\text{full}} = \frac{\pi}{3} \sum_{n=0}^{127} \Delta x \left(r_n^2 + r_n r_{n+1} + r_{n+1}^2 \right), \tag{22}$$

the standard frustum-volume quadrature summed over all slices, using the shape's own radius function $r(x)$ (Table 4) rather than the shell-based mass model of Section 2.3, since center of pressure depends on the outer aerodynamic profile, not on wall thickness.

Table 4: *Nose Radius Profile $r(x)$ by Shape (aft radius R, length L)*

Shape	$r(x)$
Conical	$R\,x/L$
Elliptical	$R\sqrt{2x/L - (x/L)^2}$
Parabolic (full)	$R\left(2x/L - (x/L)^2\right)$
Hemisphere	$R\sqrt{1 - (1 - x/L)^2}$
Tangent ogive	$\sqrt{\rho^2 - (L-x)^2} - \sqrt{\rho^2 - L^2}, \quad \rho = (L^2 + R^2)/2R$
von Kármán (Haack, $C = 0$)	$R\sqrt{\left(\theta - \frac{1}{2}\sin 2\theta\right)/\pi}, \quad \theta = \arccos(1 - 2x/L)$

Fin normal-force slope

The complete fin set's normal-force-coefficient slope (all N fins combined, in the total-angle-of-attack plane) follows the classical Barrowman fin equation for the subsonic regime ($M \leq 0.9$):

$$C_{N_\alpha,1\text{fin}} = \frac{2\pi\, s^2/A_{\text{ref}}}{1 + \sqrt{1 + (1 - M^2)\left(\dfrac{s^2}{S\cos\Gamma}\right)^2}}, \tag{23}$$

where s is fin span, A_{ref} the rocket's reference (frontal) area, S the fin planform area, and Γ the fin's mid-chord sweep angle (found from the same spanwise chord-sampling used by the drag model, Section 3.4). Summed evenly over N fins in the total-AOA plane and combined with the classical Barrowman result that this reduces to $N/2$ times the single-fin slope,

$$C_{N_\alpha,\text{fins,base}} = \frac{N}{2}\, C_{N_\alpha,1\text{fin}}. \tag{24}$$

This is then scaled by a Mach-blended body-fin interference factor K_{fB} following the fin-in-body and body-in-fin treatment of Pitts, Nielsen, and Kaattari (1959, NACA Report 1307): with $\tau = r_{\text{body}}/(s + r_{\text{body}})$,

$$K_{fB}(M) = \begin{cases} (1+\tau)^2, & M \le 0.9 \\ (1+\tau) + \left(\dfrac{1.5 - M}{0.6}\right)\tau(1+\tau), & 0.9 < M < 1.5 \\ 1+\tau, & M \ge 1.5 \end{cases} \tag{25}$$

so the fin contribution used in the rest of the model is $C_{N_\alpha,\text{fins}} = C_{N_\alpha,\text{fins,base}} \cdot K_{fB}(M)$. Green Eggs' subsonic flight regime (peak Mach of about 0.13) means the practical simulation always sits well inside the fully subsonic branch of both C_{N_α} and K_{fB}, but the model's Mach dependence is implemented in full so that the same code path is correct at any speed.

Fin center of pressure

Below Mach 0.5, the fin panel's aerodynamic center sits at the classical quarter-chord of the mean aerodynamic chord (MAC), $x_{\text{cp,fin}} = x_{\text{LE}} + x_{\text{MAC,lead}} + 0.25\,\bar{c}$. Above Mach 2, it shifts to the empirical high-speed location

$$\frac{x_{\text{cp,fin}} - x_{\text{LE}} - x_{\text{MAC,lead}}}{\bar{c}} = \frac{AR\,\beta - 0.67}{2\,AR\,\beta - 1}, \qquad \beta = \sqrt{M^2 - 1}, \tag{26}$$

with AR the fin aspect ratio. Between Mach 0.5 and 2, a fifth-order polynomial in Mach number, fit offline to match both endpoints' value and first derivative (with zero second and third derivative at the Mach 2 end), interpolates smoothly between the two regimes so that the fin CP position stays continuous and differentiable across the whole flight envelope, including transonic speeds this vehicle never actually reaches. The mean aerodynamic chord itself, its leading-edge position, and the sweep-angle cosines used above are all found by numerically sampling the fin's true planform polygon at 48 span stations rather than assuming a simple trapezoid. The same station-sampling routine (`fin_geometry_stations`) also produces the roll-sum integral used by the roll-damping model below, so both quantities stay geometrically consistent with each other and with the true freeform fin shape from the design file.

Total center of pressure and static margin

The two component center-of-pressure locations are combined by their relative contribution to total normal-force-coefficient slope, a moment-balance identity from Barrowman's original derivation:

$$x_{\text{cp,total}} = \frac{C_{N_\alpha,\text{nose}}\,x_{\text{cp,nose}} + C_{N_\alpha,\text{fins}}\,x_{\text{cp,fins}}}{C_{N_\alpha,\text{nose}} + C_{N_\alpha,\text{fins}}}. \tag{27}$$

Static margin, the distance between center of pressure and center of gravity expressed in body calibers (diameters), is computed at every simulation step from the current center of gravity (which shifts as propellant burns, Section 2.4) and this aerodynamic center of pressure:

$$\text{Margin (cal)} = \frac{x_{\text{cp,total}} - x_{\text{cg}}}{d_{\text{body}}}. \tag{28}$$

A positive static margin indicates the center of pressure sits aft of the center of gravity, meaning the rocket is statically stable: any small angle of attack produces a restoring (weathercocking) moment rather than a divergent one.

Nonlinear body lift and the fin stall clamp

The static, per-radian C_{N_α} treatment above is a small-angle linearization; it is not, by itself, evaluated with the instantaneous angle of attack α to get an actual force during the simulation. Three separate, physically distinct AOA-dependent contributions are summed instead:

1. **Nose and body linear term:** $C_{N,\text{nose}} = C_{N_\alpha,\text{nose}} \cdot \alpha$, using the raw, unclamped AOA; no stall clamp applies to this term.
2. **Galejs nonlinear body-lift term** (nose and body tube, independently): rather than a linear-in-α term, this uses the genuinely nonlinear

$$C_{N,\text{body-lift}} = K_{\text{lift}} \frac{A_{\text{planform}}}{A_{\text{ref}}} \sin\alpha \operatorname{sinc}\alpha, \qquad K_{\text{lift}} = 1.1, \tag{29}$$

   where A_{planform} is the nose's or body tube's side-view (planform) area and $\operatorname{sinc}\alpha = \sin\alpha/\alpha$ (defined as 1 at $\alpha = 0$). Because $\sin\alpha \operatorname{sinc}\alpha = \sin^2\alpha/\alpha$ is itself bounded and turns over well before α approaches 180°, this term needs no separate stall clamp, unlike an unclamped linear term.
3. **Fin term, with a 20° stall clamp:** $C_{N,\text{fins}} = C_{N_\alpha,\text{fins}} \cdot \min(\alpha, 20\check{\text{r}})$, the one true stall clamp applied during force calculation. It applies only to the fin contribution. An older, commonly cited 17.5° stall-angle constant is used only for a separate tumble-detection event check in this codebase, not as a force clamp.

The total normal force magnitude is $F_N = q\,A_{\text{ref}} \sum_i C_{N,i}$, acting in the crossflow direction (the same direction as the local lateral relative wind, unlike drag, which opposes the axial flow). A positive-margin rocket's aft fins are pushed by the crossflow like a weathervane's tail, producing a restoring moment about the CG, and the combined CP used for computing that restoring moment's moment arm is the C_N-weighted average of the three contributions' individual CP locations.

Angular-rate damping

The static CP/CG restoring-moment model above only captures the moment from a nonzero CP-CG offset. Without an additional damping term, angle-of-attack oscillations that should decay, especially in the low-dynamic-pressure region near apogee, would instead persist or grow unrealistically. Pitch and yaw damping moments are computed from the body-frame pitch and yaw rates (found by rotating the state's world-frame angular velocity into the body frame) using a damping-multiplier form combining a body-planform contribution and a fin contribution:

$$M_{\text{damp}} = 3\left[0.275\,\frac{\bar{d}_{\text{body}}\,L_{\text{body}}^4}{A_{\text{ref}}\,d} + 0.6\min(N,4)\,\frac{S_{\text{fin}}\,x_{\text{fin,mid}}^3}{A_{\text{ref}}\,d}\right]\left(\frac{\text{rate}}{v}\right)^2 q\,A_{\text{ref}}\,d, \tag{30}$$

clamped so its magnitude never exceeds the restoring moment it is damping (computed about the nose tip before the CG shift is applied, following the reference model's own convention, which never actually relocates its configurable pitch center away from the nose tip) and always applied with a sign opposing the rotation rate. This is a deliberate correction from a naive signed-clamp implementation, which was found during development to occasionally amplify the restoring moment instead of damping it under certain wind conditions, causing runaway divergence. The leading factor of 3 is an empirical tuning multiplier found to give much more realistic apogee turnover

behavior than the bare damping-multiplier formula alone. Roll damping follows an analogous per-station integral over the fin span (the same 48-station sampling used for the fin CP calculation) below a 15° local-AOA stall threshold, and a directly summed per-station stalled-chord model above it.

3.4 Drag Model

Total drag combines skin-friction drag, nose and fin pressure drag, base drag at the aft end, and transonic wave-drag corrections that activate as Mach number approaches and crosses 1, consistent with model-rocket aerodynamic practice (Niskanen, 2013):

$$C_{D,\text{total}} = C_{D,\text{body friction}} + C_{D,\text{fin friction}} + C_{D,\text{fin pressure}} + C_{D,\text{fin base}} + C_{D,\text{nose}} + C_{D,\text{base}} + C_{D,\text{wave}}. \quad (31)$$

Skin friction

The roughness-limited, Mach-corrected skin friction coefficient C_f is built from a smooth-surface base value and a surface-roughness-limited floor, taking whichever is larger, since a rough surface cannot do better than its own roughness allows no matter how the Reynolds number scales:

$$C_{f,\text{smooth}}(\text{Re}) = \begin{cases} 1.48 \times 10^{-2}, & \text{Re} < 10^4 \\ \dfrac{1}{(1.50 \ln \text{Re} - 5.6)^2}, & \text{Re} \geq 10^4 \end{cases}, \qquad C_{f,\text{rough}} = 0.032 \left(\frac{k}{L_\text{ref}} \right)^{0.2} \cdot \Xi(M), \quad (32)$$

where k is a roughness height (60 microns for Green Eggs' normal finish, from Table 5) and $\Xi(M)$ is a compressibility correction blended smoothly between Mach 0.9 and 1.1. The used coefficient is $C_f = \max(C_{f,\text{smooth}} \cdot c(M),\, C_{f,\text{rough}})$, with the Mach correction $c(M)$ itself piecewise: $c(M) = 1 - 0.1M^2$ below $M = 0.9$, transonically blended, and $c(M) = (1 + 0.15M^2)^{-0.58}$ above $M = 1.1$.

Table 5: *Surface Roughness Heights by Finish*

Finish	Roughness height k
Rough	500 microns
Rough, unfinished	250 microns
Unfinished	150 microns
Normal (Green Eggs)	60 microns
Smooth	20 microns
Polished	2 microns

This skin-friction coefficient is applied to the body via $C_{D,\text{body friction}} = C_f \,\text{FF}\,(A_{\text{wet,body}}/A_\text{ref})$, where the body form factor $\text{FF} = 1 + \frac{1}{2\,\text{FR}}$ depends on the fineness ratio $\text{FR} = L/d$, and to the fins via $C_{D,\text{fin friction}} = N\, C_f \left(1 + \frac{2t_\text{fin}}{\bar{c}}\right) \dfrac{2S_\text{fin}}{A_\text{ref}}$, meaning both sides of each fin plus a thickness correction, summed over all N fins.

Nose pressure drag

The nose pressure-drag coefficient depends on the local slope discontinuity at the nose and body-tube junction, $\sin\varphi$, sampled at 99% of the nose length rather than any single closed-form half-angle. A smoothly blending shape like Green Eggs' elliptical nose has $\sin\varphi$ near zero even though

the overall profile is not flat, correctly reflecting that there is almost no discontinuity for the airflow to see at the join:

$$C_{D,\text{nose}}(M) = \begin{cases} 0.8\sin^2\varphi, & M < 0.8 \\ 0.5\,C_{\text{base}}\,M^4 + 0.8\sin^2\varphi, & 0.8 \le M < 1.0 \\ C_{\text{base}} + 0.3\sqrt{M^2-1}, & M \ge 1.0 \end{cases} \tag{33}$$

with a shape-dependent base coefficient C_{base} (0.06 for ogive and von Kármán, 0.08 for elliptical, 0.10 conical, 0.25 hemisphere). A separate supersonic formula applies specifically to conical noses above $M = 1$: $C_D = 2.1\sin^2\varphi + 0.5\sin\varphi/\sqrt{M^2-1}$, notably dividing by $\sqrt{M^2-1}$ so the coefficient correctly plateaus toward $2.1\sin^2\varphi$ as Mach grows, the physically correct high-Mach trend. An earlier development version of this code multiplied by that factor instead, the wrong trend, and it was corrected once caught.

Base drag and wave drag

Base (aft-end) drag uses a purely Mach-dependent coefficient, $C_{D,\text{base}}(M) = \left(0.12 + 0.13M^2\right) \cdot (A_{\text{base}}/A_{\text{ref}})$ for $M \le 1$ and $0.25/M \cdot (A_{\text{base}}/A_{\text{ref}})$ above it, with no separate powered and unpowered branching (the base-drag formula used here has no engine-state dependence at all). Fin trailing-edge base drag follows the same Mach-dependent coefficient, halved for a rounded cross-section, applied per fin. Wave drag activates only above $M = 1$ (never reached in this project's flight profile, but implemented for completeness and future extensibility):

$$C_{D,\text{wave}} = K_{\text{wave}}(M) \cdot \frac{A_{\text{base}}}{A_{\text{ref}}} \cdot \frac{1}{\max(\text{FR}, 3)} \cdot \sqrt{M^2-1}, \tag{34}$$

with $K_{\text{wave}}(M) = 0.18(1 + 0.5(M-1))$ for $1 < M < 1.5$ and $0.18(1.25 + 0.1/(M-1))$ above that, additionally reduced 20% for very slender bodies ($\text{FR} > 12$).

Fin pressure drag

For a rounded or airfoil leading edge, the fin pressure-drag coefficient follows an empirical Mach-dependent curve, $(1 - M^2)^{-0.417} - 1$ below $M = 0.9$, linearly transitioning through the transonic region, and $1.214 - 0.502/M^2 + 0.1095/M^4$ above $M = 1$. For a square or blunt edge, a real stagnation-pressure coefficient $0.85\left(1 + M^2/4 + M^4/40\right)$ (subsonic) or $0.85\left(1.84 - 0.76/M^2 + 0.166/M^4 + 0.035/M^6\right)$ (supersonic) applies instead. Either way the coefficient is scaled by $\cos^2\Gamma_{\text{LE}}$ (leading-edge sweep) and applied per fin as $C_D \cdot s\,t_{\text{fin}}/A_{\text{ref}}$, summed over all fins.

3.5 Rigid-Body Dynamics and Attitude Representation

Quaternion attitude

Three-dimensional attitude is represented with a unit quaternion $\mathbf{q} = (w, x, y, z)$ mapping body-frame vectors to the world (inertial) frame, rather than Euler angles, avoiding gimbal-lock singularities and giving a numerically well-behaved orientation update via the exponential map. A vector $\mathbf{v}_{\text{body}}$ is rotated into the world frame by the sandwich product

$$\mathbf{v}_{\text{world}} = \mathbf{q} \otimes (0, \mathbf{v}_{\text{body}}) \otimes \mathbf{q}^*, \tag{35}$$

where $\mathbf{q}^* = (w, -x, -y, -z)$ is the quaternion conjugate and $\otimes$ is Hamilton quaternion multiplication. At each simulation step, the rocket's aerodynamic and thrust force vectors, computed in

the body frame, are rotated into the inertial frame using this operation before being combined with gravity. Given a rotation vector $\boldsymbol{\theta}$ (axis times angle) accumulated over a time increment, the corresponding incremental rotation quaternion is produced by the exponential map,

$$\exp\Big(\tfrac{1}{2}\boldsymbol{\theta}\Big) = \Big(\cos\tfrac{|\boldsymbol{\theta}|}{2},\ \sin\tfrac{|\boldsymbol{\theta}|}{2}\,\hat{\boldsymbol{\theta}}\Big), \tag{36}$$

which is left-multiplied onto the existing orientation quaternion to advance attitude by one time step. Angular velocity is integrated and stored in the world frame throughout this codebase, not the body frame, so the rotation-vector generator used above is the world-frame angular velocity directly, and the resulting quaternion update is a left-multiply rather than a right-multiply.

Translational dynamics

Translational acceleration follows directly from Newton's second law,

$$\dot{\mathbf{v}}_{\text{world}} = \frac{\mathbf{q} \otimes \mathbf{F}_{\text{body}} \otimes \mathbf{q}^*}{m(t)} + \mathbf{g}_{\text{world}}(h, \phi), \tag{37}$$

with gravity queried from the WGS84 model of Section 3.2 at the rocket's current altitude and the launch site's latitude on every evaluation, rather than a constant 9.80665 m/s^2, and $\mathbf{F}_{\text{body}} = \mathbf{F}_{\text{aero}} + \mathbf{F}_{\text{thrust}} + \mathbf{F}_{\text{recovery}}$ the sum of aerodynamic, motor thrust (along the body x-axis, from the motor's interpolated thrust curve), and post-deployment parachute drag forces.

Rotational dynamics

Rotational (angular) acceleration is computed in the body frame as the applied moment divided component-wise by the body's inertia tensor,

$$\boldsymbol{\alpha}_{\text{body}} = \mathbf{I}_{\text{body}}^{-1}\,\mathbf{M}_{\text{body}}, \tag{38}$$

solved as a linear system rather than a naive elementwise division since $\mathbf{I}_{\text{body}}$ is diagonal but the solve is written generally, then rotated into world coordinates for integration, since angular velocity is integrated in the world frame in this implementation:

$$\boldsymbol{\alpha}_{\text{world}} = \mathbf{q} \otimes \boldsymbol{\alpha}_{\text{body}} \otimes \mathbf{q}^*. \tag{39}$$

Consistent with the reference simulator this project targets, the gyroscopic coupling term $\boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega}$ is deliberately omitted from the rotational equation of motion rather than added for theoretical completeness, to keep the physics model directly comparable to OpenRocket's own. This is a genuine, acknowledged simplification, valid because this vehicle's roll rate stays low and its inertia is nearly axisymmetric so the omitted term stays small in practice for this specific flight, not an oversight.

3.6 Time Integration and Adaptive Step Control

The RK4 update

The 13-component state vector is advanced with the classical fixed-stage, fourth-order Runge-Kutta

scheme. Writing the state derivative function generically as $\dot{\mathbf{y}} = f(t, \mathbf{y})$,

$$\mathbf{k}_1 = f(t_n,\ \mathbf{y}_n), \tag{40}$$

$$\mathbf{k}_2 = f\Big(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{h}{2}\mathbf{k}_1\Big), \tag{41}$$

$$\mathbf{k}_3 = f\Big(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{h}{2}\mathbf{k}_2\Big), \tag{42}$$

$$\mathbf{k}_4 = f(t_n + h,\ \mathbf{y}_n + h\,\mathbf{k}_3), \tag{43}$$

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right), \tag{44}$$

with local truncation error $O(h^5)$ and global error $O(h^4)$. Position, velocity, and propellant mass are advanced this way directly component-wise. Orientation is handled slightly differently to remain on the unit-quaternion manifold: rather than naively RK4-integrating the four quaternion components as if they were independent scalars, which would not stay normalized, the same $\frac{h}{6}(\mathbf{k}_1+2\mathbf{k}_2+2\mathbf{k}_3+\mathbf{k}_4)$ weighted average is taken of the four stages' angular velocities, producing a single dt-weighted rotation vector, and the exponential map of that rotation vector is applied as one left-multiply onto the base orientation. This keeps the update exactly on the unit-quaternion manifold at every step by construction, with no separate re-normalization needed, though the implementation still normalizes defensively after each step to guard against floating-point drift.

Adaptive step-size selection

Rather than a single fixed Δt for the whole flight, the step size is chosen at every step as the minimum of eight independent candidate limits:

1. the user-requested nominal time step (0.05 s, OpenRocket's own recommended default), divided by 5 while still on the launch rail for extra resolution during the most force-sensitive phase of flight;
2. the time remaining until the next scheduled event (ignition, any motor thrust-curve sample point, burnout, or the simulation end time);
3. $\theta_{\max}/\omega_{\text{lateral}}$, limiting body rotation per step to at most 3° of combined pitch and yaw;
4. a roll-angle-step limit, capping roll rotation per step at $2 \times 28.32°$;
5. a roll-rate-change limit, capping the change in roll rate per step to 2 degrees per second;
6. a pitch and yaw rate-change limit, capping the change in pitch or yaw rate per step to 4 degrees per second;
7. while still on the rail, one-tenth of the rail length divided by current speed, so the rail-clearance event itself is resolved to fine time granularity;
8. 1.5 times the previous accepted step, preventing the step size from growing too abruptly between successive steps.

The step is additionally snapped exactly onto the next event boundary if the unconstrained choice already lands within one-twentieth of the nominal step of that boundary, and floored at a hard minimum of 1 millisecond. In practice this makes the timestep shrink dramatically around ignition, burnout, and rail clearance, where forces and rates change fastest, and relax toward the much larger nominal step during the smooth coasting and parachute-descent phases, exactly the adaptive

behavior described in the abstract. This adaptive controller, not a fixed Δt, is what let the RK4 integration match OpenRocket's own event timings (rail clear, burnout) to the millisecond in Section 5.

4. Legacy 1D Model

The project's original 1D ascent-only simulator, adapted with this rocket's real geometry and motor, compares three classical integrators (Euler, second-order Adams-Bashforth-Moulton, and fourth-order Runge-Kutta) against a simplified single-axis model of the flight, Equation 1. Because the 1D model has no recovery system and free-falls under drag after apogee, only ascent-phase metrics are compared; descent is not a fair comparison to either the full 3D simulator or OpenRocket.

4.1 Simplified Drag Model

The 1D model uses its own, simpler drag-coefficient assembly, distinct from the component-wise Barrowman and drag treatment of Section 3.4: a single lumped $C_D(v, h)$ combining body friction (with a fineness-ratio form factor $\mathrm{FF} = 1 + 60/\mathrm{FR}^3 + 0.0025\,\mathrm{FR}$), nose pressure drag, total fin friction drag, and a base-drag term with separate powered and unpowered branches. Skin friction blends a laminar Blasius form and a turbulent Prandtl-Schlichting form with a smooth sigmoid weight in $\log_{10}\mathrm{Re}$ rather than a hard Reynolds-number cutoff:

$$C_f = C_{f,\mathrm{lam}}(1-w) + C_{f,\mathrm{tur}}\,w, \qquad C_{f,\mathrm{lam}} = \frac{1.328}{\sqrt{\mathrm{Re}}}, \quad C_{f,\mathrm{tur}} = \frac{0.074}{\mathrm{Re}^{0.2}}, \quad w = \frac{1}{1 + e^{-k(\log_{10}\mathrm{Re} - \log_{10}\mathrm{Re}_t)}}, \tag{45}$$

with transition Reynolds number $\mathrm{Re}_t = 5 \times 10^5$, then compressibility corrected via $C_{f,\mathrm{comp}} = C_f\left(1 + 0.15M^2\right)^{0.58}$.

4.2 The Three Integrators

Each integrator advances the same scalar equation of motion, Equation 1, restated in explicit acceleration form as $a(v, h, t, m) = \left[T(t) - \frac{1}{2}\rho(h)C_D(v, h)S_{\mathrm{ref}}v|v| - mg\right]/m$.

Forward Euler (first-order accurate, local truncation error $O(h^2)$):

$$v_{n+1} = v_n + h\,a_n, \qquad h_{n+1} = h_n + h\,v_n. \tag{46}$$

Second-order Adams-Bashforth-Moulton (predictor-corrector). The predictor is a two-step Adams-Bashforth extrapolation using the two most recent derivative samples,

$$v^*_{n+1} = v_n + \frac{h}{2}\left(3a_n - a_{n-1}\right), \qquad h^*_{n+1} = h_n + \frac{h}{2}\left(3v_n - v_{n-1}\right), \tag{47}$$

followed by a trapezoidal (Adams-Moulton) corrector evaluated at the predicted state,

$$v_{n+1} = v_n + \frac{h}{2}\left(a^*_{n+1} + a_n\right), \qquad h_{n+1} = h_n + \frac{h}{2}\left(v^*_{n+1} + v_n\right). \tag{48}$$

Fourth-order Runge-Kutta, the same classical scheme as in Section 3.6 but applied to the scalar (h, v, m) state instead of the full rigid-body vector.

Both the ABM-2 and RK4 branches use a two-point Euler startup to bootstrap the history needed by the predictor and by the mid-step thrust and mass evaluations, respectively.

4.3 Ascent-Phase Comparison

Table 6: *1D Integrator Comparison Versus the Full 6DOF Project and Both References*

Metric	Euler (1D)	ABM-2 (1D)	RK4 (1D)	NumericalRocketry (6DOF)	OpenRocket
Apogee Altitude	99.18 m	99.23 m	99.27 m	101.30 m	101.29 m
Time to Apogee	4.69 s	4.68 s	4.68 s	4.77 s	4.77 s
Max Velocity	45.66 m/s	45.63 m/s	45.64 m/s	45.53 m/s	45.53 m/s

The three 1D integrators land within 0.1% of each other (99.18 to 99.27 m), with Euler furthest off and RK4 closest to the full 6DOF result, exactly the accuracy ordering the original comparison set out to demonstrate: the choice of numerical method matters far less here than the physics model underneath it. This is the expected theoretical ordering: Euler's first-order local truncation error accumulates fastest, ABM-2's second-order predictor-corrector improves on it, and RK4's fourth-order accuracy makes it least sensitive to the fixed $\Delta t = 0.01$ s step used by all three. The 1D model's own RK4 branch reaches apogee within 1.99% of OpenRocket's reference simulation (99.27 m vs. 101.29 m), a reasonable result for a model with no attitude dynamics, no exact Barrowman center-of-pressure treatment, and a simplified drag model. The full 6DOF project closes about 99.5% of that remaining gap, landing within 0.01% of OpenRocket, on top of adding a stability margin, off-axis motion, and a modeled recovery and descent phase the 1D model has no way to represent. Peak velocity agrees to within about 0.3% across all three integrators and the full 6DOF model alike, since peak velocity happens right at burnout, driven almost entirely by thrust and mass, the part of the flight where the extra fidelity of the 3D model matters least.

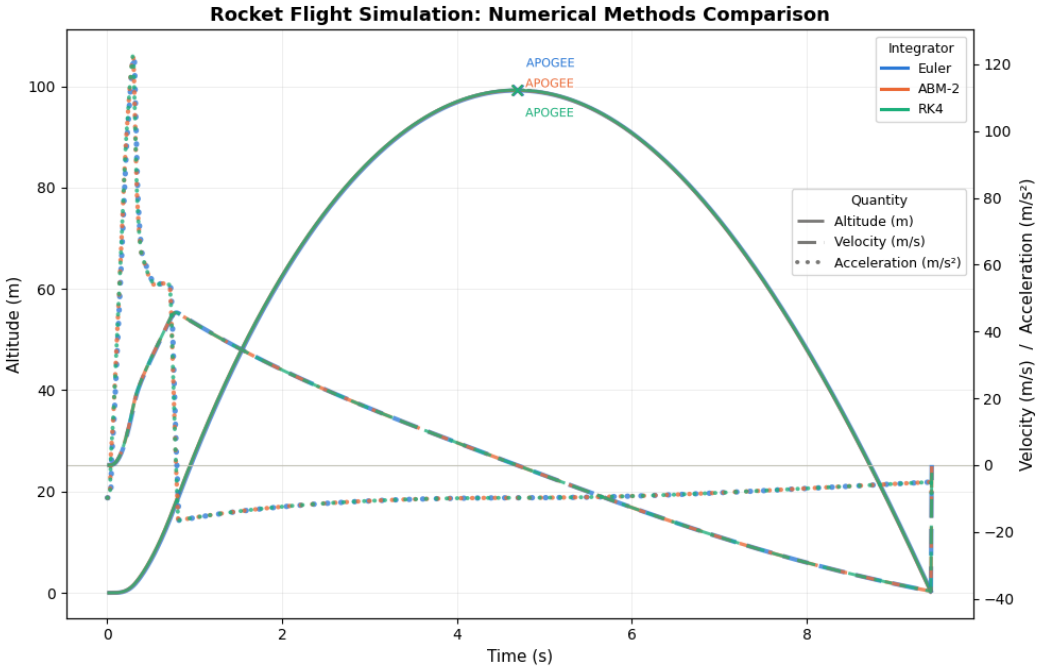

Figure 1: *Altitude, Velocity, and Acceleration Comparison Across the Three 1D Integrators*

5. Validation Results

The completed 3D simulator is validated on two independent fronts: against OpenRocket's own simulation of the identical rocket, motor, and launch conditions, and against flight-computer data recorded from an actual launch of the physical rocket.

5.1 Validation Against OpenRocket

Table 7: *Headline Results Versus OpenRocket's Reference Simulation*

Metric	NumericalRocketry	OpenRocket	Gap
Apogee Altitude	101.30 m	101.29 m	+0.01 m (0.01%)
Apogee Time	4.766 s	4.766 s	exact
Max Velocity	45.531 m/s	45.533 m/s	−0.002 m/s (0.004%)
Touchdown	27.116 s	27.194 s	−0.08 s (0.29%)

Every headline metric agrees with OpenRocket to within a few hundredths of a percent, and apogee time matches exactly to the millisecond reported by both. This is a substantially closer match than the legacy 1D model achieved (1.99% apogee error), attributable directly to the 3D model's explicit Barrowman stability treatment, full component mass model, and attitude dynamics.

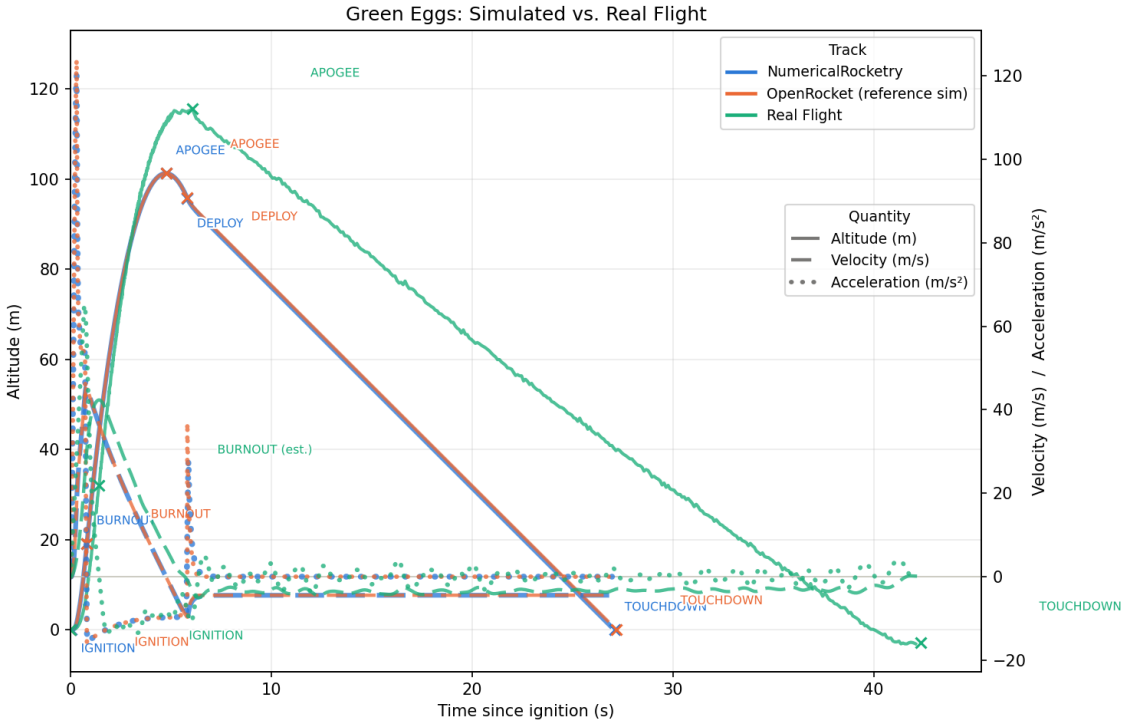

Figure 2: *Altitude vs. Time for All Three Tracks: This Project's Simulation, OpenRocket's Reference Simulation, and the Real Logged Flight*

5.2 Validation Against the Real Flight

The real flight's apogee, max velocity, and touchdown time all diverge from both simulations by more than the two simulations diverge from each other. This section covers what had to be corrected in the raw sensor log before any of these numbers were usable, the mathematical method used to do it, and what the remaining gaps mean.

Log corrections

The flight computer used for this launch, an EasyMini board (Measurement Specialties MS5607 barometric sensor), is barometer-only with no onboard accelerometer. Its raw log required four corrections before it was usable:

1. **Duplicate rows.** 66 rows in the raw 1006-row export are exact byte-for-byte repeats of their neighbor (64 of them stacked at a single $t = 0.00$ tick), a logging and export artifact rather than real samples. These were removed by direct row-equality comparison against the previous row.

2. **Wrong ground elevation.** The board's onboard altitude computation converts pressure to altitude via the standard ISA troposphere formula,

$$h_{\text{baro}}(P) = 44{,}330 \left[1 - \left(\frac{P}{P_0}\right)^{1/5.255}\right], \tag{49}$$

   using a fixed $P_0 = 1013.25$ hPa reference rather than the actual sea-level-equivalent pressure on launch day. This reads a ground elevation of 507.5 m against the site's real surveyed elevation of 532 m. The true reference pressure was recovered by a least-squares fit over every row in the flight simultaneously,

$$\hat{P}_0 = \arg\min_{P_0} \sum_i \left(h_{\text{baro}}(P_i; P_0) - h_{\text{true},i}\right)^2, \tag{50}$$

   solved with a bounded scalar minimizer, using the AGL `height` column (a pressure difference from the pad, which already cancels most of this bias) plus the known true ground elevation as the fit target. This recovered $\hat{P}_0 = 1016.23$ hPa, about 2.9 hPa above the standard reference, an ordinary day's pressure and not an outlier, and altitude and height were then recomputed directly from raw pressure using this corrected P_0.

3. **Missing pre-recording gap.** The flight computer does not begin saving samples until it is confident a real launch is underway, so the first saved row is already about 0.68 s into the flight (5 or more meters up, roughly 16 m/s). This gap is filled with a physically motivated "starts from rest" cubic $h(t) = k(t - t_0)^3$, the simplest curve consistent with $h(t_0) = 0$ and $h'(t_0) = 0$ by construction, fit to the real early-flight trend by first fitting a local quadratic to the first 0.15 s of real data and solving for the implied zero-crossing t_0. An earlier attempt to backfill this gap using the project's own 6DOF simulator was tried and rejected, since its mass and drag assumptions do not match this specific flight's real dynamics closely enough to trust for this purpose.

4. **Untrustworthy onboard speed and acceleration.** Because the EasyMini has no accelerometer, its own `acceleration`/`speed` columns are a real-time Kalman filter's estimate from barometric pressure alone. A real-time filter cannot see future samples, so it measurably lags during the fastest part of the flight, confirmed directly: integrating the onboard speed under-predicts the real height gained during boost by about 17%, a gap that stops growing once boost ends.

Velocity and acceleration re-derivation

Speed and acceleration were re-derived offline using a forward Kalman filter combined with a backward Rauch-Tung-Striebel (RTS) smoother, run as a single continuous filter across the synthetic pre-recording segment and the real recorded segment together so the two join without a seam. The state vector is $\mathbf{x} = (h, \dot{h}, \ddot{h})^\top$ (height, velocity, acceleration) evolved under a constant-jerk process model with each sample's own irregular Δt, no resampling needed, avoiding the seam artifacts a fixed-window filter such as Savitzky-Golay would introduce when the sample rate itself changes between flight phases:

$$\mathbf{x}_k = F_k\,\mathbf{x}_{k-1} + \mathbf{w}_k, \qquad F_k = \begin{pmatrix} 1 & \Delta t_k & \frac{1}{2}\Delta t_k^2 \\ 0 & 1 & \Delta t_k \\ 0 & 0 & 1 \end{pmatrix}, \tag{51}$$

with process noise covariance $Q_k = q \begin{pmatrix} \Delta t_k^5/20 & \Delta t_k^4/8 & \Delta t_k^3/6 \\ \Delta t_k^4/8 & \Delta t_k^3/3 & \Delta t_k^2/2 \\ \Delta t_k^3/6 & \Delta t_k^2/2 & \Delta t_k \end{pmatrix}$ (the standard continuous white-jerk discretization) and measurement model $z_k = H\mathbf{x}_k + v_k$, $H = (1, 0, 0)$, observing only height directly. The forward pass is the standard Kalman predict and update recursion,

$$\mathbf{x}_k^- = F_k\,\mathbf{x}_{k-1}, \qquad P_k^- = F_k P_{k-1} F_k^\top + Q_k, \tag{52}$$

$$K_k = P_k^- H^\top \left(H P_k^- H^\top + R_k\right)^{-1}, \qquad \mathbf{x}_k = \mathbf{x}_k^- + K_k\left(z_k - H\mathbf{x}_k^-\right), \tag{53}$$

$$P_k = P_k^- - K_k H P_k^-, \tag{54}$$

and the backward RTS pass then smooths every state estimate using information from later samples that the forward-only (real-time, onboard) filter could never see:

$$\mathbf{x}_k^s = \mathbf{x}_k + C_k\left(\mathbf{x}_{k+1}^s - \mathbf{x}_{k+1}^-\right), \qquad C_k = P_k F_{k+1}^\top \left(P_{k+1}^-\right)^{-1}. \tag{55}$$

Measurement noise variance R_k was estimated robustly from local-linear-fit residuals in a gentle, near-linear stretch of descent (20 to 40 s, far from any dynamic transient, so the residual scatter there is essentially pure sensor noise) and applied uniformly to the real samples, with a near-zero R assigned to the synthetic pre-recording segment, since it is an exact closed-form curve and not sensor data. This re-derivation closes most of the onboard filter's lag: 42.4 m/s versus the onboard log's originally reported 35.4 m/s, against a simulated 45.5 m/s. The first 0.3 seconds of boost remain the least certain part of this re-derivation, since the sensor's own noise there is comparable in size to the true motion, verified directly: loosening the filter's tuning there chases noise, tightening it collapses toward a biased average, with no clean middle ground that avoids both failure modes.

Apogee gap

The real flight's apogee reads about 12.42% higher than either simulation. A single real flight diverging from simulation by a few percent is a common, expected outcome in amateur rocketry validation, not necessarily a red flag: individual motors vary batch to batch within certification tolerance, an as-built rocket rarely masses exactly what its design file specifies, and launch-day atmospheric conditions are never quite the standard atmosphere both simulations assume. This project's own simulator and OpenRocket, two independently built physics engines, agree with each other to within 0.01%; for a shared modeling error to explain the real-flight gap, both would have to be wrong by about 12.42% in the same direction, which is unlikely. A barometric altimeter is

also specifically prone to dynamic-pressure error: a fast-moving rocket creates a local low-pressure region at the sensor's port (a Bernoulli effect), which a low-cost altimeter can misread as extra altitude, especially at higher speeds, consistent with the direction of the observed gap. There is nothing in this result suggesting a simulator bug; it reflects the practical limits of a low-cost flight computer rather than a modeling error.

Touchdown timing

The real flight's touchdown came roughly 56% later than either simulation (about 42 s versus about 27 s), even though apogee time and altitude are close across all three tracks. Two verified, partial contributors were identified: the real flight's onboard descent rate varies between about -1.8 and -4.2 m/s over time, confirmed directly in the raw sensor data rather than a smoothing artifact, consistent with the rocket swinging under the parachute or wind gusts affecting descent, while both simulations assume a steady terminal velocity; and the real landing site sits about 3 m below the launch pad's elevation, so the real rocket had slightly farther to fall than either simulation assumes. The elevation difference accounts for under a second of the total gap on its own; the larger remaining difference is most plausibly parachute-drag and descent-rate modeling, worth investigating further if descent modeling becomes a focus of future work.

5.3 Full Results Table

The table below reports every mission-event metric computed across all three tracks, including internal validation metrics (liftoff mass, center of gravity) that mattered during development but are not directly observable watching the rocket fly.

Event / metric	NR (sim)	OpenRocket	Real Flight	NR vs. OR	NR vs. Real
Ignition	t = 0.034 s	t = 0 s	t = 0 s[a]	n/a	n/a
Rail Clear	t = 0.256 s	t = 0.256 s	not recoverable	exact	n/a
Burnout	t = 0.81 s	t = 0.81 s	t ≈ 1.42 s[b]	exact	42.92%
Apogee Altitude	101.30 m	101.29 m	115.67 m	+0.01 m (0.01%)	−14.37 m (12.42%)
Apogee Time	4.766 s	4.766 s	6.06 s	exact	−1.294 s (21.35%)
Max Velocity	45.531 m/s	45.533 m/s	42.35 m/s[c]	−0.002 m/s (0.004%)	+3.18 m/s (7.51%)
Recovery Deploy	5.816 s	5.811 s	not detectable	5 ms	n/a
Touchdown	27.116 s	27.194 s	t = 42.36 s[d]	−0.08 s (0.29%)	−15.24 s (35.99%)
Liftoff Mass	166 g	166 g	not measured	exact	n/a
CG Location	35.52 cm	35.70 cm	not measured	−0.18 cm (0.49%)	n/a

[a]Exact by construction, the rebuilt log's own $t = 0$. [b]Estimated from the acceleration trace's zero-crossing, the same method used for the simulated tracks. [c]Re-derived offline with a Kalman filter and RTS smoother; the onboard real-time estimate reported 35.4 m/s. [d]Corrected to the point where re-derived speed first reads 0.00 m/s and stays within sensor noise, rather than the onboard flag's own 12.6 s delayed timestamp.

Burnout (42.92%) and touchdown (35.99%) look like much larger misses than they actually are:

both real-flight values are estimated or corrected using a different event definition than the simulated tracks report (burnout as thrust dropping below drag plus weight, versus total propellant consumption; touchdown as acceleration and speed settling near zero, versus the flight computer's own, much-delayed "landed" flag), so a large percentage gap here reflects that definitional difference more than actual trajectory error, unlike apogee and max velocity, which are directly comparable across all three tracks. Getting center of gravity within 0.5% of OpenRocket's own required the shape-integrated nose cone and fin set mass models described in Section 2.3, since every other metric in this table ultimately depends on that mass model being correct.

6. Discussion

Across every validated metric, the full 6DOF simulator's agreement with OpenRocket, apogee altitude within 0.01%, apogee time exact, max velocity within 0.004%, is close enough that the two can be treated as independently converging on the same underlying physics, rather than one simply copying the other's implementation details. That agreement is the strongest evidence in this project that the Barrowman aerodynamic model, WGS84 gravity, quaternion attitude integration, and adaptive RK4 time-marching scheme documented in Section 3 were implemented correctly, since OpenRocket was built independently and validated against its own separate body of flight data over many years of development. It is worth being explicit about what this agreement does and does not prove: it demonstrates that this project's independent Python reimplementation reproduces OpenRocket's own published methodology to high numerical fidelity, not that either simulator is itself a perfect model of true physical flight. That second, harder question is exactly what the real-flight comparison in Section 5.2 addresses.

The comparison against the real flight tells a different, and in some ways more valuable, story. A single real flight is one noisy sample, not a controlled repeat measurement, and the 12.42% apogee gap and 35.99% touchdown-timing gap are better explained by real-world sensor and motor variance than by a shared error in both independently built simulators. The most significant limitation exposed by this comparison is not in the flight-dynamics model at all, but in the flight computer: a barometer-only altimeter with no accelerometer cannot directly measure velocity or acceleration, and its real-time state estimate measurably lags during the fastest part of the flight. Recovering a fair, comparable real-flight track required a substantial offline re-derivation effort, a forward Kalman filter and backward RTS smoother, on top of correcting a ground-elevation error and mislabeled flight-computer state transitions, all fully documented and reproducible from the raw sensor export.

Several of the modeling choices in Section 3 were deliberate, acknowledged simplifications rather than oversights, and it is worth restating why each was made rather than fixed toward a more complete model. The omitted gyroscopic coupling term (Section 3.5) and the nose-tip-referenced damping multiplier (Section 3.3) both keep this project's physics directly comparable to the specific reference methodology it targets, rather than producing a better but no longer comparable simulation. Where a literal reproduction of that reference behavior was found during development to cause numerical pathologies, the signed damping-clamp sign-amplification bug described in Section 3.3, and the earlier supersonic nose-drag formula's wrong high-Mach trend described in Section 3.4, those were corrected, since reproducing a documented bug is not the same goal as reproducing the intended physics.

The project's central limitation is that it has exactly one real flight to validate against. Repeating the launch, ideally with a flight computer that includes an onboard accelerometer, would meaningfully strengthen the real-flight comparison and could help separate motor-to-motor variance from sensor error, neither of which this single flight can distinguish on its own.

Conclusion

This project developed and validated a Python-based 6DOF rocket flight simulator, applying numerical methods for ordinary differential equations, quaternion-based rigid-body dynamics, Barrowman aerodynamics, and adaptive Runge-Kutta integration, to a real, physically flown model rocket. The simulator's predictions agreed with OpenRocket's independently developed reference simulation to within 0.01% on apogee altitude and closed roughly 99.5% of the error remaining after the project's earlier 1D ascent model, which itself validated the underlying numerical-methods choice before the physics model was extended to full 3D. Comparison against a real logged flight surfaced a genuine, well-explained real-world limitation, barometric altimeter error under fast dynamic pressure, rather than a flaw in the simulator itself, and required a documented, reproducible sensor-correction pipeline, built on a forward Kalman filter and backward RTS smoother, to make that comparison fair in the first place.

Beyond its computational result, this project demonstrates how an undergraduate student can apply numerical methods for differential equations to build and validate a complete engineering simulation against both a reference simulator and a real physical flight, turning abstract mathematics into a practical, physically tested tool. It forms a foundation for future student research in rocket flight dynamics, including repeat flights with more capable flight instrumentation, descent and parachute-drag modeling, and extension to other rocket configurations beyond the single hardcoded vehicle this project targets.

The numerical-methods lesson of the project, established first in the 1D phase and then reconfirmed in the full 6DOF model, is that method order matters less than the underlying physics model once a scheme is at least as accurate as classical RK4: the three 1D integrators agreed with each other to within 0.1% of apogee regardless of order, while the jump from a simplified single-axis drag and thrust model to a fully three-dimensional, component-wise Barrowman treatment cut the remaining OpenRocket error by roughly two orders of magnitude. That said, RK4 combined with adaptive step control was still the right numerical choice for the 6DOF phase specifically, since a fixed timestep coarse enough to be efficient during the long coast and descent phases would have been far too coarse to resolve the millisecond-scale rail clearance and burnout events that Section 5 shows matching OpenRocket exactly. The eight-criterion step controller of Section 3.6 is what let a single integration scheme serve both regimes without the user needing to hand-tune a timestep for each flight phase.

Several concrete directions remain open for whoever continues this work. A second and third flight of the same rocket, ideally logged on a flight computer with an onboard accelerometer rather than a barometer-only board, would turn the current one-flight comparison into a real statistical sample and finally separate motor-to-motor variance from sensor error, two effects this project's single flight cannot distinguish from each other. The wind model is implemented and already threads a nonzero wind field through every stage of the aerodynamic calculation, but every validation run in this report deliberately used zero wind to match the still-air conditions actually recorded at launch; a future flight under measured nonzero wind would let that part of the model be validated for the first time rather than merely exercised. The descent and parachute-drag model is the least developed part of the physics: it uses a constant drag coefficient and area rather than the variable descent rate the real flight's own sensor log shows, and Section 5.2's touchdown-timing gap suggests that modeling parachute oscillation or a Mach- and Reynolds-dependent chute Cd could close a meaningful part of the remaining timing error. Finally, since the geometry, mass table, and motor are presently hardcoded to this one vehicle, generalizing the configuration layer to accept an

arbitrary component list would let the same validated physics core be pointed at a different rocket and motor combination without touching the underlying aerodynamics, drag, or dynamics code at all.

References

Barrowman, J. S. (1967). *The Practical Calculation of the Aerodynamic Characteristics of Slender Finned Vehicles*. Master's Thesis, The Catholic University of America.

Bryan, K. (2025). *Differential equations: A toolbox for modeling the world*. Primedia eLaunch LLC.

Derrick, W. R., & Grossman, S. I. (1997). *Elementary differential equations with boundary value problems*. Addison-Wesley.

Karbon, K. J. (1998). *Numerical methods for model rocket altitude simulation: A comparative study of accuracy and efficiency*. Apogee Rockets. `https://www.apogeerockets.com/downloads/PDFs/numeric_methods.pdf`

Niskanen, S. (2013). *OpenRocket technical documentation: Development of an Open Source model rocket simulation software*. `https://openrocket.sourceforge.net/techdoc.pdf`

Niskanen, S. (2015). *OpenRocket* (Version 15.03) [Computer software]. `https://github.com/openrocket/openrocket`

Pitts, W. C., Nielsen, J. N., & Kaattari, G. E. (1959). *Lift and Center of Pressure of Wing-Body-Tail Combinations at Subsonic, Transonic, and Supersonic Speeds*. NACA Report 1307.